

Lab 10: Spring WebFlux & WebClient

วิชา: CP353002 หลักการออกแบบและพัฒนาซอฟต์แวร์

ชื่อ-สกุล: นายภควัฒน์ สุขมณี

รหัสนักศึกษา: 673380418-9 | Section : 4

1. Reactive Programming vs Blocking I/O

หัวข้อ	Blocking I/O (Imperative)	Reactive Programming (Non-blocking)
รูปแบบการทำงาน	Thread per Request: 1 Request จะถือครอง 1 Thread และหยุดรอ (Block) จนกว่า I/O เช่น Database/Network จะตอบกลับ	Asynchronous & Event-driven: ใช้ Thread จำนวนน้อย (Event Loop) ส่ง Request แล้วไปทำงานอื่นต่อทันที เมื่อได้ข้อมูลกลับมาจึงแจ้ง Event
การจัดการ Resource	สิ้นเปลือง Memory และ CPU เมื่อมี Concurrent Request จำนวนมาก เพราะ Threads ต้องรอกัน (Thread Pool Exhaustion)	ประหยัด Resource สูง รองรับ High Concurrency ได้อย่างมีประสิทธิภาพ
Return Type	คืนค่าเป็นวัตถุตรงๆ เช่น Product, List<Product>	คืนค่าเป็น Publisher Stream เช่น Mono<T>, Flux<T>
เหมาะสำหรับ	แอปพลิเคชัน CRUD ทั่วไปที่มี Traffic ปานกลาง	ระบบ High-concurrency, Real-time Streaming, Microservices ที่คุยกันผ่าน HTTP

2. Mono vs Flux

ทั้ง Mono และ Flux เป็นคลาสที่มาจาก interface `Publisher<T>` ใน Project Reactor โดยมีความแตกต่างในการใช้งานดังนี้:

2.1 Mono<T>

ความหมาย: Data Stream ที่ส่งคืนข้อมูล 0 หรือ 1 รายการ (Single/Empty Value) กรณีที่เลือกใช้: ใช้กับงานที่ทราบแน่นอนว่าผลลัพธ์มีเพียงรายการเดียวหรือว่างเปล่า เช่น: การค้นหาตาม Primary Key เช่น `findById(id)` การบันทึกหรืออัปเดตข้อมูล เช่น `save(entity)` Operation ที่ไม่มีการคืนค่าข้อมูล เช่น `deleteById(id)` ซึ่งจะคืนค่าเป็น `Mono<Void>`

เหตุผล: เหมาะสมกับนิยามของ Single-value Async Operations ช่วยให้ประเภทข้อมูล (Type Signature) บ่งบอกพฤติกรรมชัดเจน

2.2 Flux<T>

ความหมาย: Data Stream ที่ส่งคืนข้อมูล 0 ถึง N รายการ (Multiple/Sequence Values) กรณีที่เลือกใช้: ใช้กับงานที่มีผลลัพธ์เป็นชุดข้อมูล หลายรายการ หรือ Stream แบบไม่สิ้นสุด เช่น: การดึงข้อมูลทั้งหมด เช่น `findAll()` การค้นหาข้อมูลตามเงื่อนไข เช่น `findByCategory(category)` การรับส่งข้อมูลแบบ Real-time Event Streaming (SSE / WebSockets)

เหตุผล: ออกแบบมาเพื่อจัดการข้อมูลแบบ Non-blocking Collection โดยประมวลผลข้อมูลแต่ละ Item ที่เข้ามาใน Stream ทันทีโดยไม่ต้องรอให้ได้ข้อมูลครบทั้งหมดเหมือน `List<T>`

3. WebClient Method Chain Explanation

WebClient เป็น Non-blocking HTTP Client สำหรับเรียกใช้ REST API แบบ Reactive โดยมีลำดับ Method Chain สำคัญ ดังนี้ :

`.get() / .post() / .delete()`

หน้าที่: กำหนด HTTP Method ของ Request ที่ต้องการส่งไปยัง Remote Server

`.uri(String uri, Object... uriVariables)`

หน้าที่: กำหนด Endpoint Path และใส่ค่า Path Variables หรือ Query Parameters เช่น `/products/{id}`

`.retrieve()`

หน้าที่: สั่ง Execute ส่ง Request และรับ Response Body เพื่อเตรียมนำมา แปลง/Extract ข้อมูลในขั้นตอนถัดไป

`.bodyToMono(Class<T> elementClass) / .bodyToFlux(Class<T> elementClass)`

หน้าที่: แปลง Response Body ที่ได้รับมาจาก HTTP Response ให้กลายเป็น Reactive Data Stream

ใช้ `.bodyToMono(...)` เมื่อคาดหวังผลลัพธ์ 1 รายการ (เช่น `Product.class` หรือ `Void.class`)

ใช้ `.bodyToFlux(...)` เมื่อคาดหวังผลลัพธ์หลายรายการ (เช่น `Product.class`)

4. Source Code & Explanations

4.1 Repository Layer (ProductRepository.java) ทำหน้าที่จำลองการเข้าถึงข้อมูลแบบ In-Memory Async

package com.example.lab10.repository;
import com.example.lab10.model.Product;
import org.springframework.stereotype.Repository;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
@Repository
public class ProductRepository {
// ใช้ ConcurrentHashMap จำลองฐานข้อมูล เพื่อรองรับการทำงานแบบ Concurrent
private final Map<String, Product> database = new ConcurrentHashMap<>();
public ProductRepository() {
// Seed ข้อมูลเริ่มต้น โดยใส่ชื่อและรหัสนักศึกษาในข้อมูลสินค้า
database.put("1", new Product("1", "Laptop (673380418-9)", "Electronics", "Apple", 10, 35000.0, "MEMBER"));
database.put("2", new Product("2", "Mouse", "Electronics", "Logitech", 50, 850.0, "NONE"));
database.put("3", new Product("3", "Coffee Mug", "Home", "IKEA", 20, 250.0, "SEASONAL"));
}
// ค้นหาตาม ID คืนค่าเป็น Mono<Product> (0 หรือ 1 ค่า)
public Mono<Product> findById(String id) {
return Mono.justOrEmpty(database.get(id));
}
// ดึงข้อมูลทั้งหมด คืนค่าเป็น Flux<Product> (0 ถึง N ค่า)
public Flux<Product> findAll() {
return Flux.fromIterable(database.values());

}
// บันทึกข้อมูลสินค้า
public Mono<Product> save(Product product) {
database.put(product.getId(), product);
return Mono.just(product);
}
// ลบข้อมูลสินค้าตาม ID คืนค่า Mono<Void> เมื่อสำเร็จ
public Mono<Void> deleteById(String id) {
database.remove(id);
return Mono.empty();
}
// กรองสินค้าตาม Category โดยใช้ Operator filter
public Flux<Product> findByCategory(String category) {
return Flux.fromIterable(database.values())
.filter(product -> product.getCategory().equalsIgnoreCase(category));
}
}

4.2 Service Layer (ProductService.java) ทำหน้าที่จัดการ Business Logic และส่งต่อ Reactive Data Stream

package com.example.lab10.service;
import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@Service
public class ProductService {
private final ProductRepository repository;
public ProductService(ProductRepository repository) {
this.repository = repository;
}
public Mono<Product> getByid(String id) {
return repository.findById(id);
}
public Flux<Product> getAll() {
return repository.findAll();
}
public Mono<Product> create(Product product) {
return repository.save(product);
}
public Mono<Void> delete(String id) {
return repository.deleteByid(id);
}

public Flux<Product> getByCategory(String category) {
return repository.findByCategory(category);
}
// ใช้ map Operator เพื่อแปลง Stream ของ Product ให้เป็น Double (ดึงเฉพาะราคา)
public Mono<Double> getPriceById(String id) {
return repository.findById(id)
.map(Product::getPrice);
}
}

4.3 Controller Layer (ProductController.java) สร้าง Non-blocking REST Endpoints

package com.example.lab10.controller;
import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@RestController
@RequestMapping("/products")
public class ProductController {
private final ProductService service;
public ProductController(ProductService service) {
this.service = service;
}
// GET /products/{id} -> ดึงสินค้า 1 รายการ
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
return service.getById(id);
}
// GET /products -> ดึงรายการสินค้าทั้งหมด
@GetMapping
public Flux<Product> getAll() {
return service.getAll();
}
// POST /products -> สร้างสินค้าใหม่
@PostMapping

public Mono<Product> create(@RequestBody Product product) {
return service.create(product);
}
// DELETE /products/{id} -> ลบสินค้าตาม ID
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
return service.delete(id);
}
// GET /products/category/{cat} -> ค้นหาสินค้าตามหมวดหมู่
@GetMapping("/category/{cat}")
public Flux<Product> getByCategory(@PathVariable("cat") String category) {
return service.getByCategory(category);
}
// GET /products/{id}/price -> ดึงเฉพาะราคาสินค้าตาม ID
@GetMapping("/{id}/price")
public Mono<Double> getPrice(@PathVariable String id) {
return service.getPriceById(id);
}
}

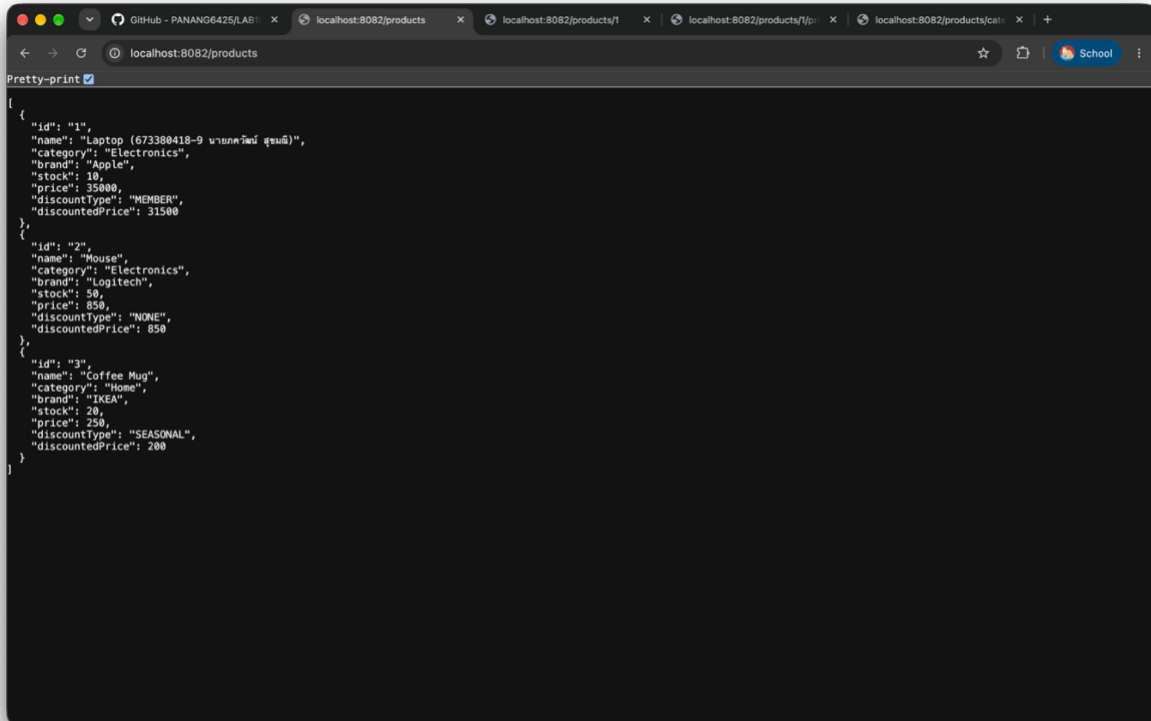
4.4 WebClient Layer (ProductWebClient.java)implementation การเรียกใช้ HTTP Endpoints แบบ Non-blocking

package com.example.lab10.client;
import com.example.lab10.model.Product;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@Component
public class ProductWebClient {
private final WebClient webClient;
public ProductWebClient(WebClient.Builder builder) {
this.webClient = builder.baseUrl("http://localhost:8082").build();
}
// GET /products/{id}
public Mono<Product> getProductById(String id) {
return webClient.get()
.uri("/products/{id}", id)
.retrieve()
.bodyToMono(Product.class);
}
// GET /products
public Flux<Product> getAllProducts() {
return webClient.get()
.uri("/products")
.retrieve()
.bodyToFlux(Product.class);
}

// POST /products
public Mono<Product> createProduct(Product product) {
return webClient.post()
.uri("/products")
.bodyValue(product)
.retrieve()
.bodyToMono(Product.class);
}
// DELETE /products/{id}
public Mono<Void> deleteProduct(String id) {
return webClient.delete()
.uri("/products/{id}", id)
.retrieve()
.bodyToMono(Void.class);
}
// GET /products/category/{cat}
public Flux<Product> getProductsByCategory(String category) {
return webClient.get()
.uri("/products/category/{cat}", category)
.retrieve()
.bodyToFlux(Product.class);
}
// GET /products/{id}/price
public Mono<Double> getProductPrice(String id) {
return webClient.get()
.uri("/products/{id}/price", id)
.retrieve()
.bodyToMono(Double.class);
}
}

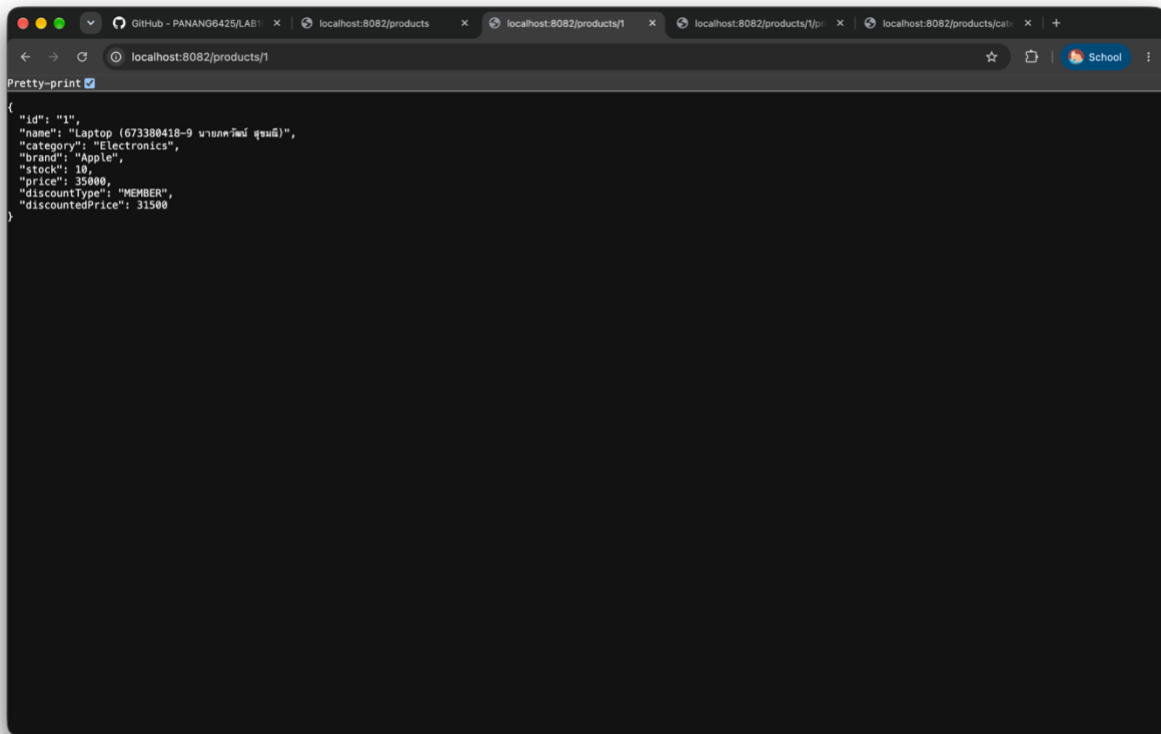
5 การทดสอบ END POINT

5.1 GET /products

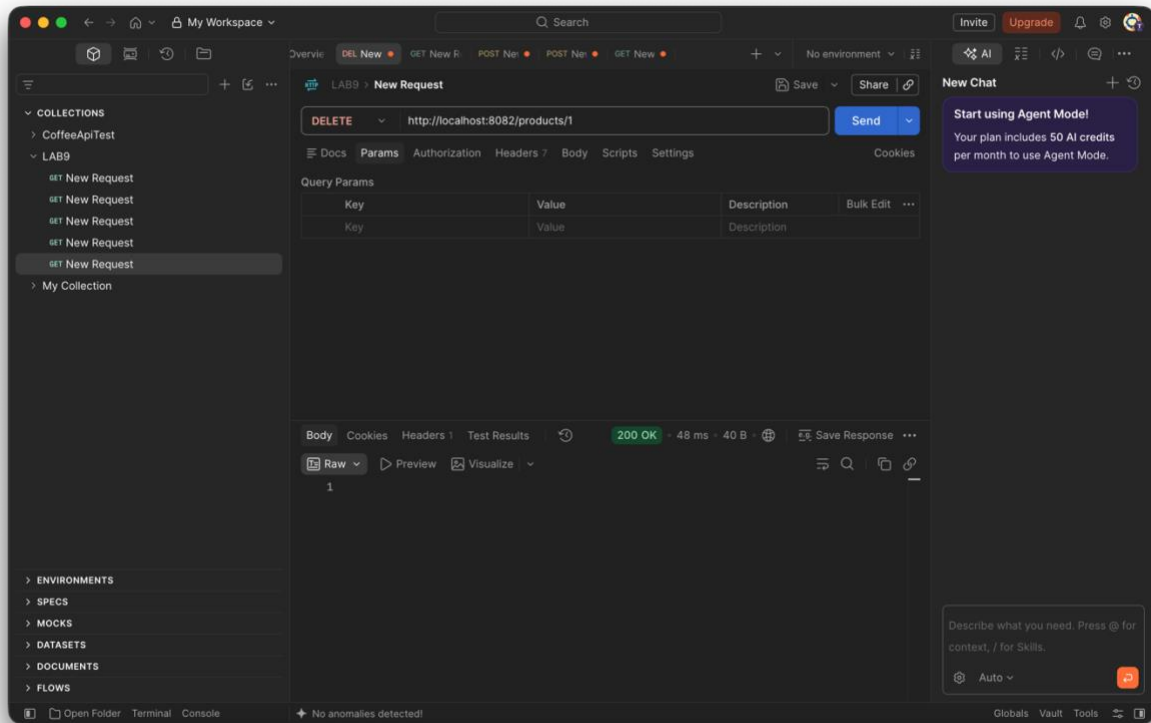


```
{
  [
    {
      "id": "1",
      "name": "Laptop (673380418-9 นายภควัฒน์ สุระรัมย์)",
      "category": "Electronics",
      "brand": "Apple",
      "stock": 10,
      "price": 35000,
      "discountType": "MEMBER",
      "discountedPrice": 31500
    },
    {
      "id": "2",
      "name": "Mouse",
      "category": "Electronics",
      "brand": "Logitech",
      "stock": 50,
      "price": 850,
      "discountType": "NONE",
      "discountedPrice": 850
    },
    {
      "id": "3",
      "name": "Coffee Mug",
      "category": "Home",
      "brand": "IKEA",
      "stock": 20,
      "price": 250,
      "discountType": "SEASONAL",
      "discountedPrice": 200
    }
  ]
}
```

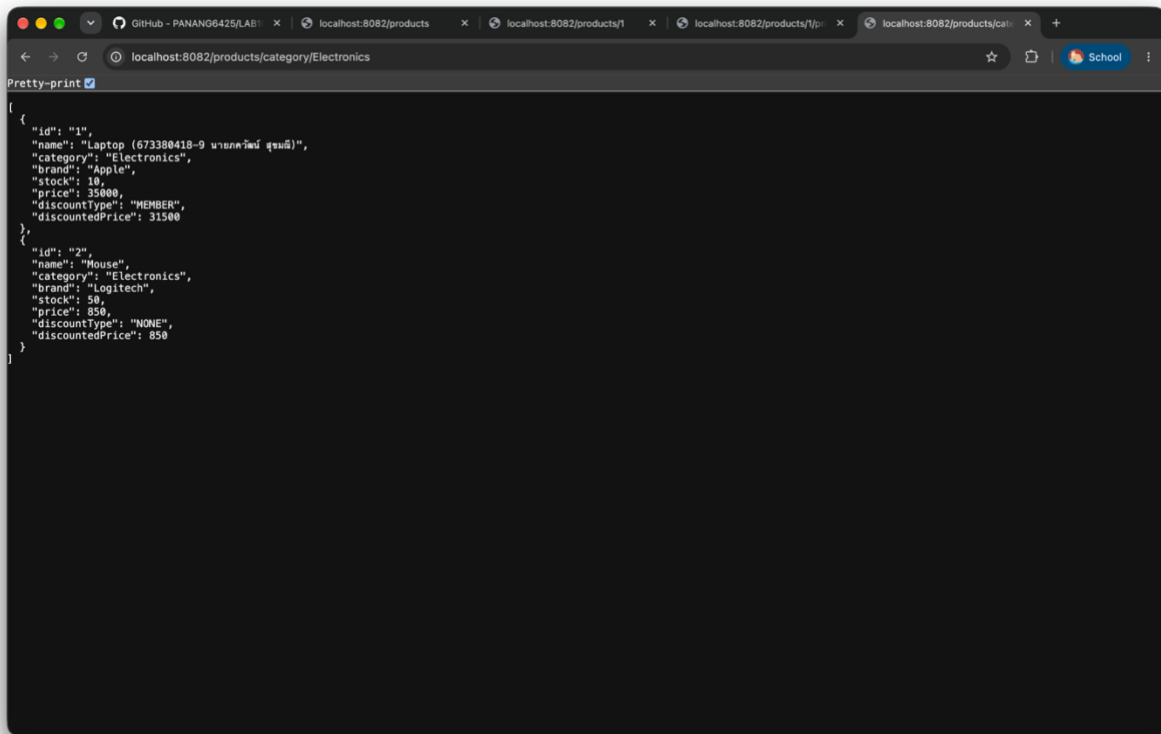
5.2 POST/products



5.3 DELETE/products/{id}



5.4 GET/products/category/{ELC}



The screenshot shows a web browser window with the address bar displaying `localhost:8082/products/category/Electronics`. The page content is a JSON array of two product objects, displayed in a dark-themed editor with a 'Pretty-print' button. The first object represents a laptop with a 'MEMBER' discount, and the second represents a mouse with no discount.

```
{
  "id": "1",
  "name": "Laptop (673380418-9 นานาเทคโนโลยี รุ่นดี)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 10,
  "price": 35000,
  "discountType": "MEMBER",
  "discountedPrice": 31500
},
{
  "id": "2",
  "name": "Mouse",
  "category": "Electronics",
  "brand": "Logitech",
  "stock": 50,
  "price": 850,
  "discountType": "NONE",
  "discountedPrice": 850
}
```

5.5 GET/products/{id}/price

